Absolutely — below is a **clean, Claude-friendly Markdown spec**, rewritten from scratch, **updated to reflect the refactors you actually implemented** in the EBNF (jobs list split, scoped document sources, zip enforcement, PDF split variants, Apple/Google commented out, etc.).

This is written as a **handoff document** you can drop directly into Claude Code so it has clear context, intent, rules, and known risks.

Claude Code Handoff Spec

C2M Jobs Submit API (Simplified Endpoints)

EBNF → OpenAPI Pipeline

---

0. Context & Goals

We are transitioning to a **simplified Jobs submission API** under:

```
bash

POST /jobs/submit/...
```

The design goal is:

- Make **common workflows simple**
- Preserve **advanced workflows** (zip, merge, PDF split, address capture)
- Keep the **EBNF Data Dictionary (DD) as the single source of truth**
- Ensure the existing **EBNF → OpenAPI translator** can successfully generate a usable OpenAPI YAML
- Treat OpenAPI, Postman collections, and tests as **derived artifacts**

This document explains:

- What endpoints exist
- How the EBNF is structured
- What rules are expressed in grammar vs comments
- Known translator risks and mitigations
- What Claude Code should validate or refactor (if needed)

---

1. Primary Artifacts

1.1 EBNF Data Dictionary (Source of Truth)

Baseline file:

```
csharp

jobs-requests-dd-baseline-with-endpoint-rules.ebnf
```

Contains:

- Endpoint **`submit...Params`** rules
- Reusable object rules (**`docSourceStandard`**, **`recipientAddressSource`**, etc.)
- Endpoint-level decision rules in comments
- Translator-friendly refactors already applied

  Important:
  Apple Pay and Google Pay are intentionally **commented out** (both in **`paymentDetails`** alternation and in their full definitions).
  This is deliberate for initial testing and integration stability.

1.2 Generated OpenAPI YAML

- Produced automatically by CI/CD from the EBNF
- Used to:
  - Generate or import Postman collections
  - Drive request validation
  - Form the public API contract (once stabilized)

---

2. Simplified Endpoints (Current Model)

2.1 Most Common

`POST /jobs/submit/single/doc`

- Single document submission
- Supports:
  - Single recipient
  - Inline address list (mail-merge style)
  - Stored address lists
- Uses **`docSourceAll`**

`POST /jobs/submit/single/pdf/addressCapture`

- Single PDF
- **No recipientAddressSource in body**
- Addresses captured via external workflow

- Uses `docSourceStandard`

2.2 PDF Split Workflows

`POST /jobs/submit/single/pdf/split`

- One PDF split into multiple segments
- Each split segment **must include recipientAddressSource**
- Uses:
  - `pdfSplitJobsWithAddress`
  - `docSourceStandard`

`POST /jobs/submit/single/pdf/split/addressCapture`

- Same split mechanics
- **No recipientAddressSource allowed**
- Uses:
  - `pdfSplitJobsNoAddress`
  - `docSourceStandard`

2.3 Multi-Job Workflows

`POST /jobs/submit/multi/doc`

- Multiple jobs in one request
- Each job includes:
  - `documentSource`
  - `recipientAddressSource`
- Uses:
  - `multiDocJobs`

`POST /jobs/submit/multi/doc/merge`

- Merge multiple documents, then submit as **one job**
- Requires:
  - `mergeDocumentSource`
  - `recipientAddressSource`
- Server must enforce ≥2 documents

`POST /jobs/submit/multi/zip`

- Bulk jobs referencing files inside a zip
- **Schema-level zip enforcement**
- Uses:
  - `multiZipJobs`
  - `docSourceZipFile` (zip-only)

`POST /jobs/submit/multi/zip/addressCapture`

- Zip-based workflow
- **No jobs[]**
- **No recipientAddressSource**
- Addresses captured externally
- Uses:
  - `zipDocumentSource`

---

3. Purpose of the Documentation

This documentation exists to support a **strict pipeline**:

1. **EBNF defines request shapes** (source of truth)
2. CI/CD translates EBNF → OpenAPI YAML
3. OpenAPI YAML is used to:
   - Generate Postman collections
   - Generate tests
   - Validate requests

Core Principle

> **EBNF defines structure, not runtime behavior.**
> Conditional logic lives in:
>
> - Endpoint-level comments
> - Server-side validation
> - (Optionally) OpenAPI constraints added post-generation

---

4. Structural Concepts Used in the EBNF

4.1 Grammar Dialect (Legacy-Compatible)

- Rule definition:

```ini
ruleName = ... ;
```

- Object composition:

```css
A + B + C
```

- Optional:

```css
[ field ]
```

- Alternatives:

```less
A | B
```

- Arrays:

```nginx
{ item }
```

(Always 0..N in grammar; minItems enforced elsewhere)

4.2 Scoped Document Sources (Translator-Friendly)

Atomic primitives (unchanged):

- **`documentIdSource`**
- **`requestIdSource`**
- **`urlSource`**
- **`zipDocumentIdSource`**
- **`zipRequestIdSource`**

Scoped groupings (important refactor):

```ebnf
docSourceStandard = documentIdSource | requestIdSource | urlSource ;
docSourceZipFile  = zipDocumentIdSource | zipRequestIdSource ;
docSourceAll      = docSourceStandard | docSourceZipFile ;
```

This prevents endpoints from being **too permissive** in generated OpenAPI.

4.3 Job List Refactor (Critical)

We intentionally **removed the shared `jobs = { job }` union**.

Instead:

```
ebnf

multiDocJobs = { multiDocJobItem } ;

pdfSplitJobsWithAddress = { pdfSplitJobItemWithAddress } ;
pdfSplitJobsNoAddress   = { pdfSplitJobItemNoAddress } ;
```

This allows:

- Stronger OpenAPI schemas
- Endpoint-specific validation
- Less ambiguity for the translator

---

5. Endpoint-Level Decision Rules (Must Be Enforced)

These rules **cannot be fully expressed in EBNF** and must be implemented in server logic.

Each endpoint in the DD includes detailed comments describing these rules.

A. `documentSource` Runtime Rules

- If **`requestIdSource`** is used:
  - **`filename`** required **only** if upload request had multiple files
- For zip flows:
  - **`zipFilename`** selects the zip
  - **`filename`** selects an entry inside the zip

B. Address Capture Endpoints

These **must reject inline addresses**:

- **`/single/pdf/addressCapture`**
- **`/single/pdf/split/addressCapture`**
- **`/multi/zip/addressCapture`**

C. jobs[] Constraints

- All jobs arrays must be **non-empty**

- Item type is endpoint-specific:
  - PDF split → split job items only
  - Multi doc / zip → multi job items only

D. `jobTemplate` vs `jobOptions`

Recommended policy (documented):

- Reject requests containing **both**
- If both absent:
  - Either default or reject (explicit decision required)

E. Top-Level vs Per-Job `jobTemplate`

For `multi/doc` and `multi/zip`:

- If top-level `jobTemplate` present:
  - Applies to all jobs
- Per-job override:
  - Either explicitly allowed **or**
  - Explicitly rejected
    (Must be documented and enforced)

---

6. Payment Model (Current State)

Active Variants

- `creditCardPayment`
- `invoicePayment`
- `achPayment`
- `userCreditPayment`

Intentionally Disabled (Commented Out)

- Apple Pay
- Google Pay

Both the **alternation lines** and **full definitions** are commented out.

This keeps the tagged-union structure intact while reducing integration scope.

---

7. Known Translator Risks & Mitigations

1. Arrays Are 0..N in Grammar

Affected rules:

- `multiDocJobs`
- `pdfSplitJobs*`
- `addressList`
- `documentsToMerge`

Mitigation:
Enforce `minItems` via server validation or OpenAPI patching.

---

2. Tagged Union Translation (paymentDetails)

String-literal discriminators may not translate cleanly.

Current approach:
Keep existing model; Apple/Google disabled.

Fallback if needed:
Refactor to explicit discriminator + payload.

---

3. tags Shape Ambiguity

Current model may generate:

```
json

"tags": [ "a", "b" ]
```

If API requires:

```
json

"tags": { "tags": [...] }
```

Then EBNF must be refactored to an explicit wrapper.

---

4. Merge Requires ≥2 Documents

EBNF cannot express this.

Mitigation:
Document and enforce in server logic.

---

8. What Claude Code Is Asked To Do

1. **Run the EBNF → OpenAPI translator**

2. Inspect generated OpenAPI for:
    - job list specificity
    - documentSource scoping
    - paymentDetails correctness
    - tags shape
3. Identify any translator failures or schema degradation
4. Propose **minimal refactors only if necessary**
5. Produce a short **Translator Risk Report**:
    - What failed
    - Why
    - What refactor fixed it

---

9. Summary

- The EBNF has already been refactored for translator friendliness
- Endpoint semantics are clearer and stricter
- Remaining complexity is intentionally handled via:
    - Comments
    - Validation logic
- Apple/Google Pay are deferred by design

This spec should give Claude Code everything needed to:

- Understand intent
- Validate correctness
- Safely evolve the pipeline without semantic drift

If you want, next we can:

- Add a **"Do Not Refactor" section** for Claude
- Generate a **translator smoke-test checklist**
- Or produce a **before/after OpenAPI diff guide**

Just say the word.